

Лабораторная работа № 3

Тема: Параметры маршрутов, query-параметры и тело запроса

Цель: Научиться получать данные HTTP-запроса через `req.params`, `req.query` и `req.body`, создавать динамические маршруты, фильтровать данные с помощью query-параметров и принимать JSON-данные для создания и изменения ресурсов.

Практическое задание

Продолжить разработку проекта **Internet Shop API**.

В предыдущих работах сервер уже содержал маршруты для товаров, категорий и заказов. Теперь необходимо сделать маршруты динамическими и реализовать реальную работу с массивом товаров. Используйте массив минимум из пяти товаров:

```
let products = [  
  {  
    id: 1,  
    name: "Keyboard",  
    price: 15000,  
    category: "Accessories",  
    inStock: true  
  },  
  {  
    id: 2,  
    name: "Mouse",  
    price: 8000,  
    category: "Accessories",  
    inStock: true  
  },  
  {  
    id: 3,  
    name: "Monitor",  
    price: 85000,  
    category: "Monitors",  
    inStock: false  
  },  
  {  
    id: 4,  
    name: "Laptop",  
    price: 420000,  
    category: "Computers",  
    inStock: true  
  },  
  {  
    id: 5,  
    name: "Webcam",  
    price: 22000,  
  }  
]
```

```
    category: "Accessories",  
    inStock: false  
  }  
];
```

Последовательность выполнения

1. Подключите обработку JSON.

После создания объекта app добавьте:

```
app.use(express.json());
```

Middleware должен располагаться **до маршрутов**, использующих req.body.

2. Реализуйте получение конкретного товара.

Создайте маршрут:

GET /api/products/:id

Например:

GET /api/products/3

должен вернуть товар с id = 3.

Используйте:

```
const id = Number(req.params.id);
```

и метод массива:

```
products.find(...)
```

Если товар отсутствует, сервер должен вернуть:

```
{  
  "message": "Product not found"  
}
```

со статусом:

404 Not Found

3. Добавьте фильтрацию по категории.

Маршрут:

GET /api/products?category=Accessories

должен возвращать только товары соответствующей категории.

Если параметр category отсутствует:

GET /api/products

сервер должен возвращать весь массив.

4. Добавьте фильтрацию по наличию.

Поддержите запросы:

GET /api/products?inStock=true

GET /api/products?inStock=false

Не забывайте, что значение query-параметра приходит как строка.

Преобразование можно выполнить так:

```
const inStock = req.query.inStock === "true";
```

5. Разрешите использовать несколько фильтров одновременно.

Запрос:

GET /api/products?category=Accessories&inStock=true

должен вернуть только товары, которые одновременно:

- относятся к категории Accessories;
- находятся в наличии.

Фильтры должны применяться последовательно к одному массиву result.

6. Добавьте фильтрацию по диапазону цен.

Поддержите параметры:

minPrice

maxPrice

Пример:

GET /api/products?minPrice=10000&maxPrice=50000

Сервер должен возвращать товары стоимостью от 10000 до 50000.

Преобразуйте полученные значения в числа.

7. Реализуйте создание товара.

Создайте маршрут:

POST /api/products

Клиент должен передавать JSON:

```
{
  "name": "Headphones",
  "price": 35000,
  "category": "Accessories",
  "inStock": true
}
```

Сервер должен сформировать новый объект с идентификатором и добавить его в массив.

Например:

```
const newProduct = {
  id: products.length + 1,
  ...req.body
};
```

```
products.push(newProduct);
```

Верните созданный объект со статусом:

201 Created

8. Проверьте содержимое req.body.

Временно добавьте:

```
console.log(req.body);
```

Отправьте POST-запрос через HTTP-клиент.

Убедитесь, что объект из JSON действительно появляется в терминале.

После проверки console.log() можно удалить.

9. Реализуйте частичное изменение товара.

Создайте маршрут:

PATCH /api/products/:id

Например:

PATCH /api/products/3

с телом:

```
{  
  "price": 79000,  
  "inStock": true  
}
```

должен изменить только эти свойства товара №3.

Для поиска используйте req.params.id.

Для получения изменяемых данных используйте req.body.

Если товар не найден, верните статус 404.

10. Реализуйте удаление товара по идентификатору.

Создайте:

DELETE /api/products/:id

Пример:

DELETE /api/products/2

Сервер должен удалить товар №2 из массива.

Можно найти индекс:

```
const index = products.findIndex(  
  product => product.id === id  
);
```

и затем удалить элемент:

```
products.splice(index, 1);
```

Если товар отсутствует, верните 404.

11. Создайте маршрут демонстрации всех источников данных.

Создайте:

PATCH /api/products/:id/settings

Поддержите запрос:

PATCH /api/products/4/settings?notify=true

с телом:

```
{  
  "price": 399000  
}
```

В ответ сервер должен вернуть объект:

```
{  
  "id": 4,  
  "notify": true,  
  "changes": {  
    "price": 399000  
  }  
}
```

При этом:

req.params.id используется для id;

req.query.notify используется для notify;

req.body используется для changes.

Этот маршрут нужен для демонстрации одновременной работы всех трёх источников данных.

Обязательные запросы для проверки

Проверьте HTTP-клиентом:

GET /api/products

GET /api/products/3

GET /api/products/999

GET /api/products?category=Accessories

GET /api/products?inStock=true

GET /api/products?category=Accessories&inStock=true

GET /api/products?minPrice=10000&maxPrice=50000

POST /api/products

PATCH /api/products/3

DELETE /api/products/2

PATCH /api/products/4/settings?notify=true

Требования к результату

Приложение должно:

- использовать express.json();
- получать динамический id через req.params;
- преобразовывать числовой id из строки в число;
- возвращать конкретный товар по идентификатору;
- возвращать 404, если товар отсутствует;
- использовать req.query для фильтрации;
- поддерживать минимум три query-параметра;
- поддерживать одновременное применение нескольких фильтров;
- принимать JSON через req.body;
- создавать новый товар;
- частично изменять товар;
- удалять товар по идентификатору;
- содержать пример совместного использования params, query и body.